

Solução acordada pelo grupo para resolução da Pilha Rubro Negra

Alunos:
Débora Samara
Lucas Gabriel
Robson Douglas
Victor Passos
Walber Ranniere

```
public class PilhaRubroNegra {
    // Variáveis
    public int capacidade;
    public Object[] a;
    public int tVermelho, tPreto;

    // Construtor
    public PilhaRubroNegra(int capacidade) {
        this.capacidade = capacidade;
        tVermelho = -1;
        tPreto = capacidade;
        a = new Object[capacidade];
    }

    public int capacidade() {
        return capacidade;
    }

    public void aumentarPilha() {
        Object[] array_aux = new Object[capacidade * 2];

        // Copia pilha vermelha
        for (int i = 0; i < sizeV(); i++) {
            array_aux[i] = a[i];
        }

        // Novo topo da pilha preta
        int novoTPreto = capacidade * 2 - sizeP();

        // Copia pilha preta corretamente
        for (int j = 0; j < sizeP(); j++) {
            array_aux[novoTPreto + j] = a[tPreto + j];
        }

        a = array_aux;
        capacidade = capacidade * 2;
        tPreto = novoTPreto;
    }
}
```

```

public void diminuirPilha() {
    Object[] array_aux = new Object[capacidade / 2];

    // Copia pilha vermelha
    for (int i = 0; i < sizeV(); i++) {
        array_aux[i] = a[i];
    }

    int novoTPreto = (capacidade / 2) - sizeP();

    // Copia pilha preta corretamente
    for (int j = 0; j < sizeP(); j++) {
        array_aux[novoTPreto + j] = a[tPreto + j];
    }

    a = array_aux;
    capacidade = capacidade / 2;
    tPreto = novoTPreto;
}

public void verificaCheio() {
    if ((sizeV() + sizeP()) == capacidade) {
        aumentarPilha();
    }
}

public void verificaUmTerco() {
    if ((sizeV() + sizeP()) <= capacidade / 3) {
        diminuirPilha();
    }
}

public int sizeV() {
    return tVermelho + 1;
}

public int sizeP() {
    return capacidade - tPreto;
}

public boolean isEmptyV() {
    return tVermelho == -1;
}

public boolean isEmptyP() {
    return tPreto == capacidade;
}

```

```

public Object topV() {
    if (isEmptyV()) {
        throw new RuntimeException("A pilha vermelha tá vazia!");
    }
    return a[tVermelho];
}

public Object topP() {
    if (isEmptyP()) {
        throw new RuntimeException("A pilha preta tá vazia!");
    }
    return a[tPreto];
}

public void pushV(Object o) {
    a[++tVermelho] = o;
    verificaCheio();
}

public void pushP(Object o) {
    a[--tPreto] = o;
    verificaCheio();
}

public Object popV() {
    if (isEmptyV()) {
        throw new RuntimeException("A pilha vermelha tá vazia!!");
    }
    Object r = a[tVermelho--];
    verificaUmTerco();
    return r;
}

public Object popP() {
    if (isEmptyP()) {
        throw new RuntimeException("A pilha preta tá vazia!!");
    }
    Object l = a[tPreto++];
    verificaUmTerco();
    return l;
}
}

```